FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	RAMOS, Nadine Love D.
Section:	AN43
Professor:	Crisola Tan
Date:	October 2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**

(Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}
```

```
# -------- Run Model ----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation ----------
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**

    - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.

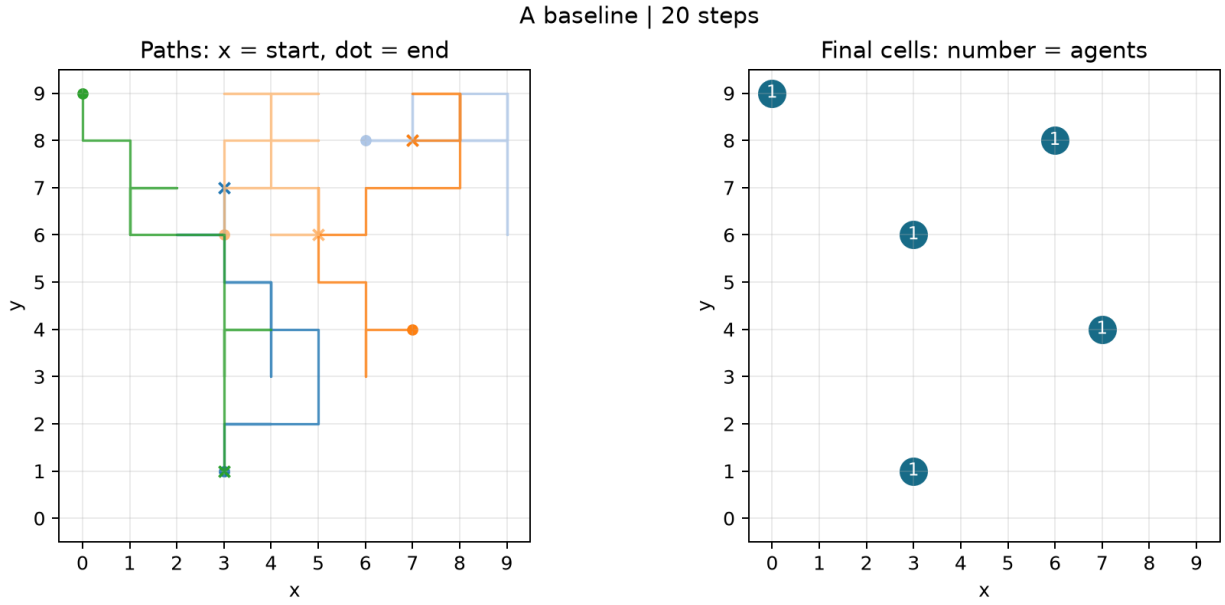
Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks)⇩

A. Baseline

5 agents | 10 x 10 grid | 20 steps | random movement | seed 65

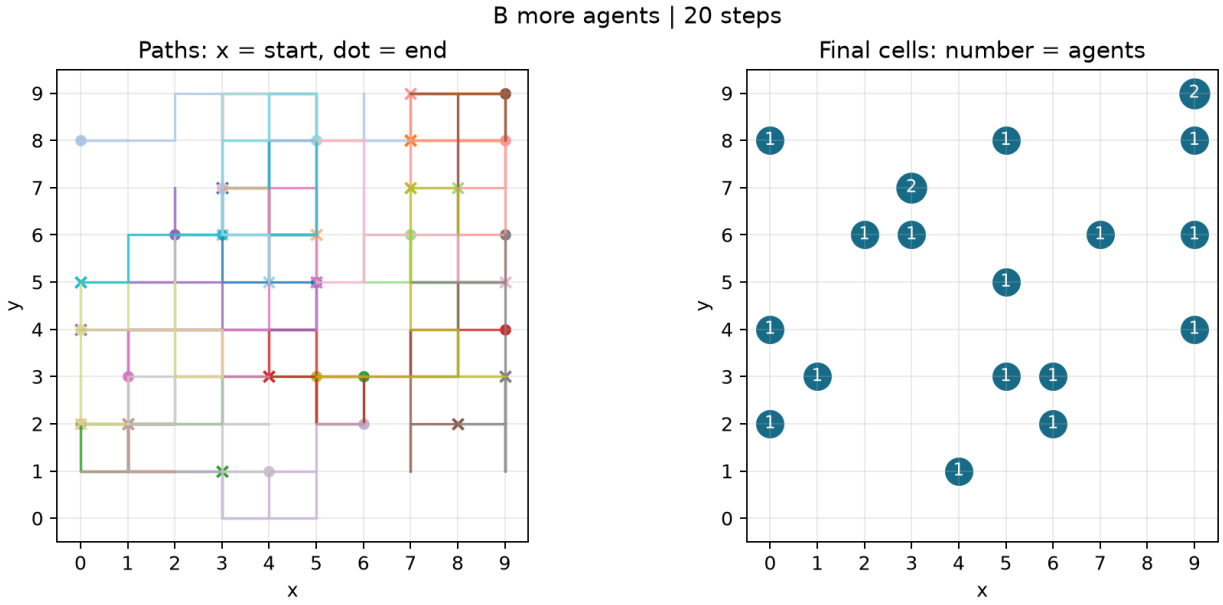



```
Scenario: A_baseline
Agents: 5 | Grid: 10 x 10
Steps: 20 | Rule: random | Seed: 65
Agent | Start (x, y) | Final (x, y)
    1 | ( 3,  7)     | ( 3,  1)
    2 | ( 7,  8)     | ( 6,  8)
    3 | ( 7,  8)     | ( 7,  4)
    4 | ( 5,  6)     | ( 3,  6)
    5 | ( 3,  1)     | ( 0,  9)
Cells visited by all agents: 49
Different final cells: 5
Final cells shared by agents: 0
Mean distance from start (grid steps): 4.80
Mean change in x: -1.20
```

Observation: The five agents ended in five different cells. Their paths cross, but no cell is shared at the end. Agent 5 reached (0, 9). All agents stayed within coordinates 0 to 9.

B. More agents

20 agents | 10 x 10 grid | 20 steps | random movement | seed 65

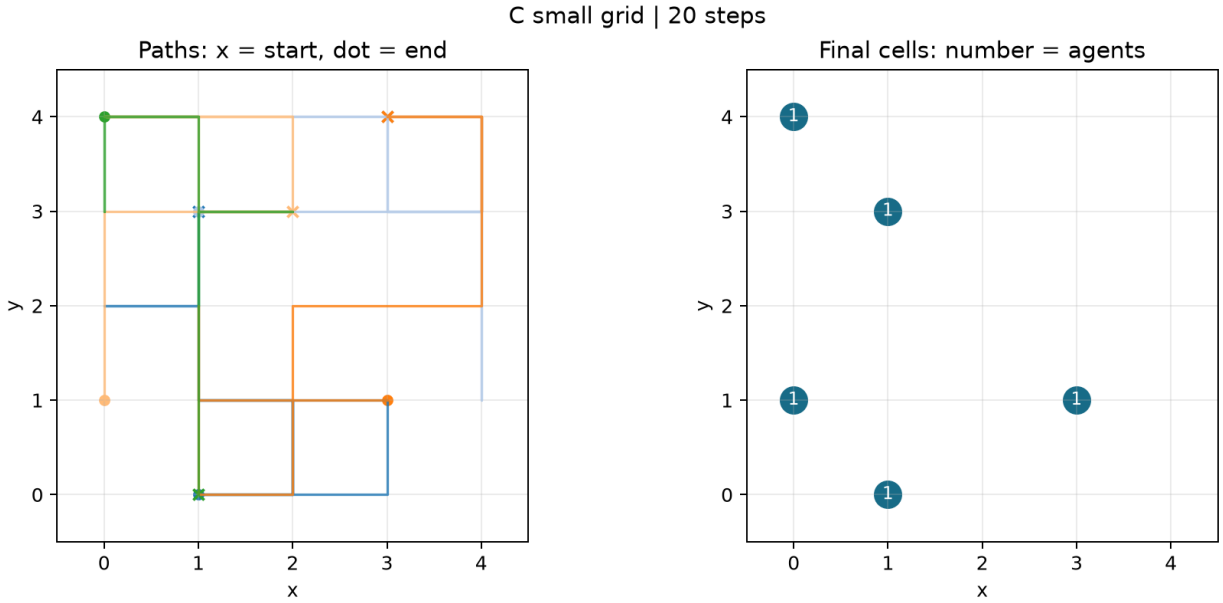



```
Scenario: B_more_agents
Agents: 20 | Grid: 10 x 10
Steps: 20 | Rule: random | Seed: 65
Agent | Start (x, y) | Final (x, y)
    1 | ( 3,  7)     | ( 5,  5)
    2 | ( 7,  8)     | ( 0,  8)
    3 | ( 7,  8)     | ( 9,  9)
    4 | ( 5,  6)     | ( 3,  7)
    5 | ( 3,  1)     | ( 6,  3)
    6 | ( 8,  7)     | ( 7,  6)
    7 | ( 4,  3)     | ( 9,  4)
    8 | ( 7,  9)     | ( 9,  8)
    9 | ( 0,  4)     | ( 2,  6)
   10 | ( 1,  2)     | ( 6,  2)
   11 | ( 8,  2)     | ( 9,  9)
   12 | ( 1,  2)     | ( 0,  2)
   13 | ( 5,  5)     | ( 1,  3)
   14 | ( 9,  5)     | ( 3,  7)
   15 | ( 9,  3)     | ( 9,  6)
   16 | ( 3,  6)     | ( 4,  1)
   17 | ( 7,  7)     | ( 5,  3)
   18 | ( 0,  2)     | ( 0,  4)
   19 | ( 0,  5)     | ( 3,  6)
   20 | ( 4,  5)     | ( 5,  8)
Cells visited by all agents: 85
Different final cells: 18
Final cells shared by agents: 2
Mean distance from start (grid steps): 4.50
Mean change in x: 0.20
```

Observation: The 20 agents ended in 18 cells. Two cells are shared: (9, 9) and (3, 7), with two agents each. More paths overlap than in the baseline.

C. Small grid

5 agents | 5 x 5 grid | 20 steps | random movement | seed 65

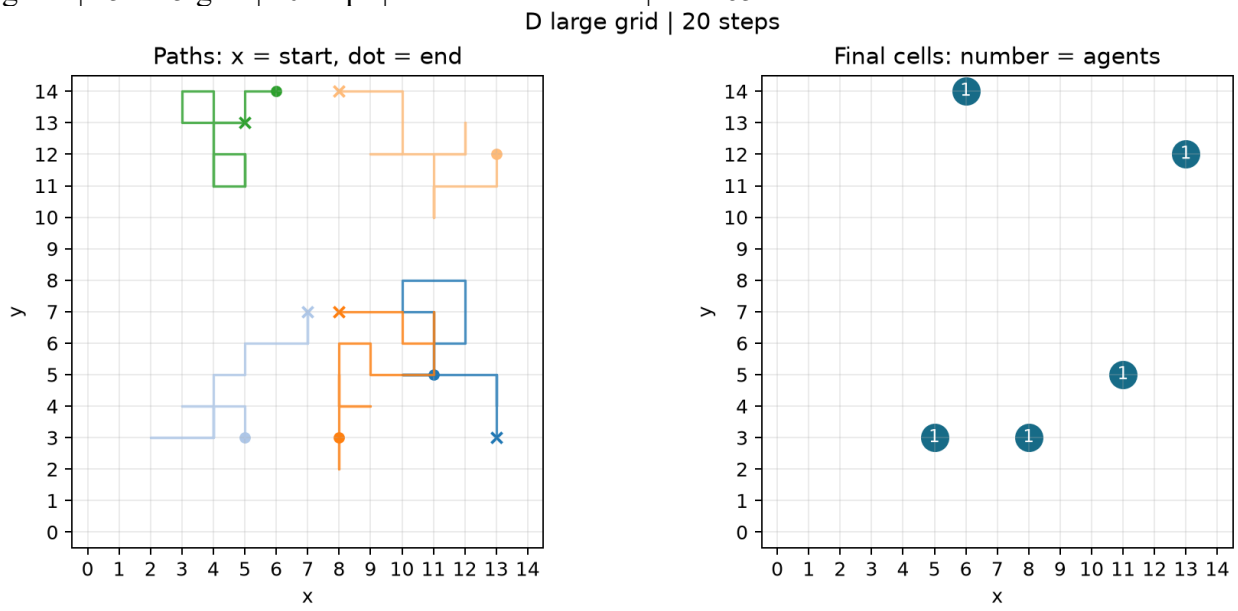



```
Scenario: C_small_grid
Agents: 5 | Grid: 5 x 5
Steps: 20 | Rule: random | Seed: 65
Agent | Start (x, y) | Final (x, y)
    1 | ( 1,  3)     | ( 1,  0)
    2 | ( 3,  4)     | ( 1,  3)
    3 | ( 3,  4)     | ( 3,  1)
    4 | ( 2,  3)     | ( 0,  1)
    5 | ( 1,  0)     | ( 0,  4)
Cells visited by all agents: 23
Different final cells: 5
Final cells shared by agents: 0
Mean distance from start (grid steps): 3.60
Mean change in x: -1.00
```

Observation: The agents visited 23 of the 25 cells. They move through much of this small grid. They ended in five different cells, so sharing a final cell is not guaranteed.

D. Large grid

5 agents | 15 x 15 grid | 20 steps | random movement | seed 65

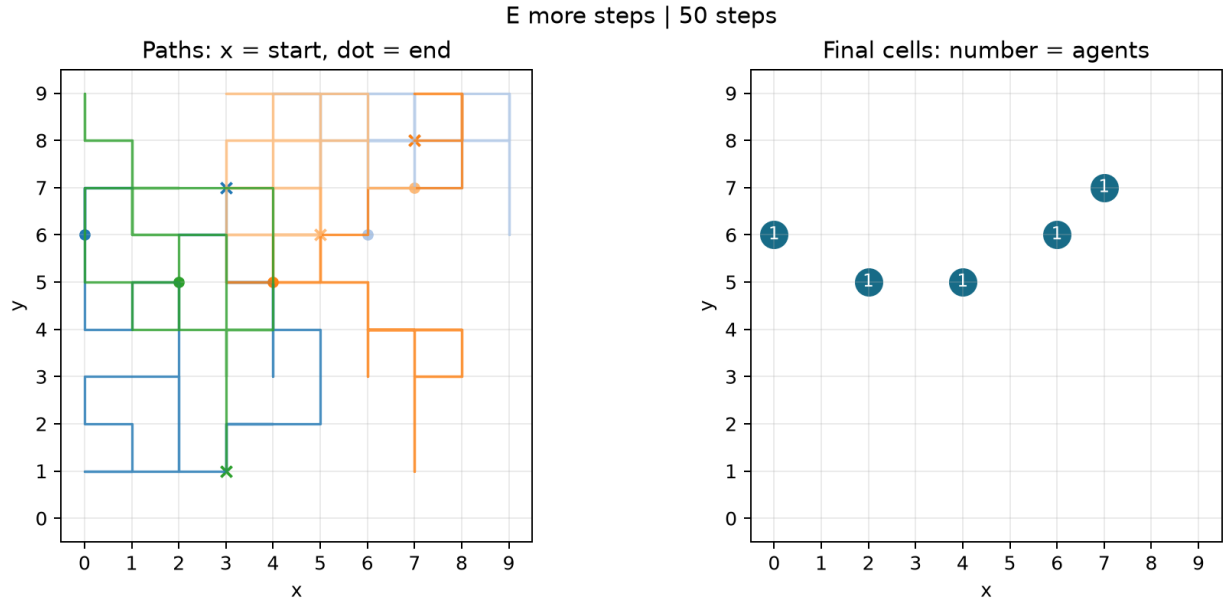



```
Scenario: D_large_grid
Agents: 5 | Grid: 15 x 15
Steps: 20 | Rule: random | Seed: 65
Agent | Start (x, y) | Final (x, y)
    1 | (13,  3)     | (11,  5)
    2 | ( 7,  7)     | ( 5,  3)
    3 | ( 8,  7)     | ( 8,  3)
    4 | ( 8, 14)     | (13, 12)
    5 | ( 5, 13)     | ( 6, 14)
Cells visited by all agents: 63
Different final cells: 5
Final cells shared by agents: 0
Mean distance from start (grid steps): 4.60
Mean change in x: 0.40
```

Observation: The agents visited 63 of the 225 cells. Their final positions are spread across a larger area, and many cells remain unvisited. No final cell is shared.

E. More steps

5 agents | 10 x 10 grid | 50 steps | random movement | seed 65

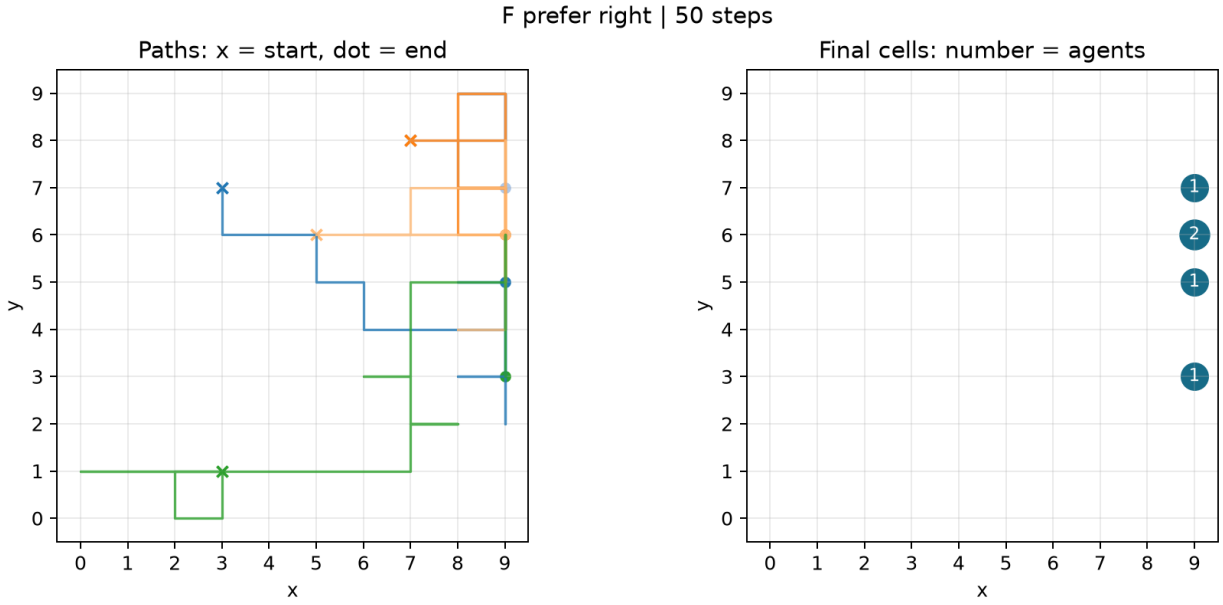



```
Scenario: E_more_steps
Agents: 5 | Grid: 10 x 10
Steps: 50 | Rule: random | Seed: 65
Agent | Start (x, y) | Final (x, y)
    1 | ( 3,  7)     | ( 0,  6)
    2 | ( 7,  8)     | ( 6,  6)
    3 | ( 7,  8)     | ( 4,  5)
    4 | ( 5,  6)     | ( 7,  7)
    5 | ( 3,  1)     | ( 2,  5)
Cells visited by all agents: 72
Different final cells: 5
Final cells shared by agents: 0
Mean distance from start (grid steps): 4.20
Mean change in x: -1.20
```

Observation: The agents visited 72 cells, compared with 49 after 20 steps. Their mean distance from the start fell from 4.80 to 4.20 grid steps because random paths can turn back.

F. Prefer right

5 agents | 10 x 10 grid | 50 steps | right movement | seed 65




```
Scenario: F_prefer_right
Agents: 5 | Grid: 10 x 10
Steps: 50 | Rule: right | Seed: 65
Agent | Start (x, y) | Final (x, y)
    1 | ( 3,  7)     | ( 9,  5)
    2 | ( 7,  8)     | ( 9,  7)
    3 | ( 7,  8)     | ( 9,  6)
    4 | ( 5,  6)     | ( 9,  6)
    5 | ( 3,  1)     | ( 9,  3)
Cells visited by all agents: 42
Different final cells: 4
Final cells shared by agents: 1
Mean distance from start (grid steps): 5.40
Mean change in x: 4.00
```

Observation: All five agents ended at the right edge, x = 9. Agents 3 and 4 share (9, 6). The mean change in x is +4.00, compared with -1.20 under the random rule for 50 steps.

Compare the scenarios

Case	Agents	Grid	Steps	Visited	Final cells
A	5	10 x 10	20	49	5
B	20	10 x 10	20	85	18
C	5	5 x 5	20	23	5
D	5	15 x 15	20	63	5
E	5	10 x 10	50	72	5
F	5	10 x 10	50	42	4

Visited: Different cells visited by any agent, including the starting positions.
Final cells: Different cells occupied at the end.

1. More agents: A compared with B
Only the number of agents changes, from 5 to 20. The group visits more cells (49 to 85), and two final cells are shared in B. The grid looks busier.

2. Grid size: A compared with C and D
All three use 5 agents and 20 steps. The small grid has 23 of 25 cells visited (92%). The baseline has 49 of 100 (49%). The large grid has 63 of 225 (28%). The larger grid has more space, so the agents cover a smaller part of it in this run.

3. More steps: A compared with E
Only the step count changes, from 20 to 50. E continues the same starting paths used in A. The agents visit 23 more cells. They do not all finish farther from their starts because they can return to earlier places.

4. Movement rule: E compared with F
Both use 5 agents, a 10 x 10 grid, 50 steps, and the same starting positions. Only the movement rule changes. Random movement leaves agents in different parts of the grid. The right rule puts every final position at x = 9. It visits fewer cells (42 instead of 72) because movement gathers near one edge.

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

Answer the Questions ⇩

1. What happens if the number of agents is very high compared to the grid size?
The grid becomes crowded. Paths cross more often, and several agents may share a cell. In this program they can overlap, so crowding does not stop them from moving. If there are more agents than cells, at least one cell must be shared. With an avoidance rule, crowded agents would have fewer free places to move.

2. How does increasing the number of steps affect the spread of agents?
More steps give agents more chances to explore. In these runs, the visited cells increased from 49 at 20 steps to 72 at 50 steps. However, random steps can bring agents back toward their starts. Their mean final distance fell from 4.80 to 4.20, so more steps do not always mean a wider final spread. The grid boundary also limits how far they can go.

3. Do the agents ever move outside the grid? Why or why not?
No. The code keeps x and y between 0 and grid size minus 1. On a 10 x 10 grid, both values must stay from 0 to 9. If an agent at an edge tries to move outward, it stays in its current cell for that step. It does not wrap around to the other side.

4. How can this simulation model real-life phenomena, such as diffusion or crowd movement?
An agent can stand for a small particle. Random steps give a simple picture of particles spreading through an area, such as a smell spreading in a room. An agent can also stand for a person. A preferred direction can represent people heading toward an exit. A more realistic crowd model would need walls, exits, and rules that stop people from walking through each other.

Complete Python program

TA1_RAMOS.py

```
"""CS0065 TA1: random walks, saved paths, and scenario comparisons."""

import argparse
import csv
import json
import os
from collections import Counter
from pathlib import Path

ROOT = Path(__file__).resolve().parent
os.environ.setdefault("MPLCONFIGDIR", str(ROOT / ".mplconfig"))

import agentpy as ap
import matplotlib

# Save plots and animations without requiring a desktop window.
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation, PillowWriter

DIRECTIONS = [(1, 0), (-1, 0), (0, 1), (0, -1)]


class RandomWalker(ap.Agent):
    def setup(self):
        self.path = []

    def step(self):
        # Right mode gives right a 60% chance; other directions get 40%.
        weights = [0.6, 0.1, 0.15, 0.15]
        if self.p.rule == "random":
            weights = [0.25] * 4
        dx, dy = self.model.random.choices(DIRECTIONS, weights=weights)[0]
        x, y = self.model.grid.positions[self]
        limit = self.p.size - 1
        # Clamp each coordinate so an attempted outside move stays inside.
        position = (max(0, min(limit, x + dx)),
                    max(0, min(limit, y + dy)))
        self.model.grid.move_to(self, position)
        self.path.append(position)


class RandomWalkModel(ap.Model):
    def setup(self):
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)
        self.grid = ap.Grid(self, (self.p.size, self.p.size), torus=False)
        positions = [(self.random.randrange(self.p.size),
                      self.random.randrange(self.p.size))
                     for _ in self.agents]
        # Sharing a cell is allowed, as in the starter movement rule.
        self.grid.add_agents(self.agents, positions=positions)
        for agent in self.agents:
            agent.path.append(self.grid.positions[agent])

    def step(self):
```

```
        self.agents.step()


def simulate(count, size, steps, rule="random", seed=65):
    if min(count, size, steps) < 1:
        raise ValueError("Agents, grid size, and steps must be at least 1.")
    if rule not in ("random", "right"):
        raise ValueError("Movement rule must be random or right.")
    params = dict(agents=count, size=size, steps=steps, rule=rule)
    model = RandomWalkModel(params)
    model.run(seed=seed, display=False)
    return model


def save_csv(path, header, rows):
    with path.open("w", newline="", encoding="utf-8") as file:
        writer = csv.writer(file)
        writer.writerow(header)
        writer.writerows(rows)


def configure_grid(ax, size, title):
    ax.set(xlim=(-0.5, size - 0.5), ylim=(-0.5, size - 0.5),
           xlabel="x", ylabel="y", title=title, aspect="equal")
    ticks = range(0, size, max(1, size // 15))
    ax.set_xticks(ticks)
    ax.set_yticks(ticks)
    ax.grid(alpha=0.25)


def save_outputs(model, name, seed, animate=False):
    folder = ROOT / "outputs" / name
    folder.mkdir(parents=True, exist_ok=True)
    paths = [agent.path for agent in model.agents]
    rows = [(i, t, x, y) for i, path in enumerate(paths, 1)
            for t, (x, y) in enumerate(path)]
    save_csv(folder / "paths.csv", ["agent", "step", "x", "y"], rows)
    final = [(i, *path[0], *path[-1])
             for i, path in enumerate(paths, 1)]
    save_csv(folder / "final_positions.csv",
             ["agent", "start_x", "start_y", "final_x", "final_y"], final)
    occupied = Counter(path[-1] for path in paths)
    visited = set(position for path in paths for position in path)
    distance = [abs(p[-1][0] - p[0][0]) + abs(p[-1][1] - p[0][1])
                for p in paths]
    shift = sum(p[-1][0] - p[0][0] for p in paths) / len(paths)
    summary = dict(name=name, agents=len(paths), size=model.p.size,
                   steps=model.p.steps, rule=model.p.rule, seed=seed,
                   visited_cells=len(visited), final_cells=len(occupied),
                   shared_cells=sum(n > 1 for n in occupied.values()),
                   mean_distance=sum(distance) / len(paths),
                   mean_x_shift=shift, final_positions=final)
    lines = [f"Scenario: {name}",
             f"Agents: {len(paths)} | Grid: {model.p.size} x {model.p.size}",
             f"Steps: {model.p.steps} | Rule: {model.p.rule} | Seed: {seed}",
             "Agent | Start (x, y) | Final (x, y)"]
    lines += [f"{i:5} | ({sx:2}, {sy:2})     | ({x:2}, {y:2})"
              for i, sx, sy, x, y in final]
```

```
    lines += [f"Cells visited by all agents: {len(visited)}",
              f"Different final cells: {len(occupied)}",
              f"Final cells shared by agents: {summary['shared_cells']}",
              f"Mean distance from start (grid steps): {summary['mean_distance']:.2f}",
              f"Mean change in x: {shift:.2f}"]
    text = "\n".join(lines)
    (folder / "console.txt").write_text(text + "\n", encoding="utf-8")
    (folder / "summary.json").write_text(
        json.dumps(summary, indent=2), encoding="utf-8")
    print(text + "\n")

    fig, axes = plt.subplots(1, 2, figsize=(10, 4.4), layout="constrained")
    colors = [plt.get_cmap("tab20")(i % 20) for i in range(len(paths))]
    for path, color in zip(paths, colors):
        xs, ys = zip(*path)
        axes[0].plot(xs, ys, color=color, alpha=0.8, linewidth=1.3)
        axes[0].scatter(xs[0], ys[0], color=color, marker="x", s=32)
        axes[0].scatter(xs[-1], ys[-1], color=color, s=24)
    for (x, y), number in occupied.items():
        axes[1].scatter(x, y, s=150 + 40 * number, color="#176b87")
        axes[1].text(x, y, str(number), ha="center", va="center",
                     color="white", fontsize=9)
    configure_grid(axes[0], model.p.size, "Paths: x = start, dot = end")
    configure_grid(axes[1], model.p.size, "Final cells: number = agents")
    fig.suptitle(name.replace("_", " ") + f" | {model.p.steps} steps")
    fig.savefig(folder / "plot.png", dpi=180)
    plt.close(fig)

    if animate:
        fig, ax = plt.subplots(figsize=(6, 6), layout="constrained")
        configure_grid(ax, model.p.size, "Recorded agent paths")
        trails = [ax.plot([], [], color=c, linewidth=1)[0] for c in colors]
        dots = [ax.plot([], [], "o", color=c)[0] for c in colors]

        def update(frame):
            # Replay saved paths; drawing a frame never moves an agent.
            for path, line, dot in zip(paths, trails, dots):
                xs, ys = zip(*path[:frame + 1])
                line.set_data(xs, ys)
                dot.set_data([xs[-1]], [ys[-1]])
            ax.set_title(f"{name}: step {frame} of {model.p.steps}")
            return trails + dots

        movie = FuncAnimation(fig, update, frames=model.p.steps + 1,
                              interval=180, blit=False, repeat=False)
        movie.save(folder / "animation.gif", writer=PillowWriter(fps=6))
        plt.close(fig)
    return summary


def positive_input(prompt):
    while True:
        try:
            value = int(input(prompt))
            if value > 0:
                return value
        except ValueError:
            pass
```

```
        print("Please enter a whole number greater than 0.")


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--demo", action="store_true",
                        help="Run the six report scenarios.")
    parser.add_argument("--animate", action="store_true",
                        help="Also save an animated GIF for each run.")
    parser.add_argument("--seed", type=int, default=65)
    args = parser.parse_args()
    if args.demo:
        cases = [("A_baseline", 5, 10, 20, "random"),
                 ("B_more_agents", 20, 10, 20, "random"),
                 ("C_small_grid", 5, 5, 20, "random"),
                 ("D_large_grid", 5, 15, 20, "random"),
                 ("E_more_steps", 5, 10, 50, "random"),
                 ("F_prefer_right", 5, 10, 50, "right")]
    else:
        count = positive_input("Enter number of agents: ")
        size = positive_input("Enter grid size (10 means 10 x 10): ")
        steps = positive_input("Enter number of steps: ")
        rule = input("Movement rule (random/right) [random]: ").strip().lower()
        while rule not in ("", "random", "right"):
            rule = input("Please enter random or right: ").strip().lower()
        cases = [("custom", count, size, steps, rule or "random")]
    summaries = []
    for name, count, size, steps, rule in cases:
        model = simulate(count, size, steps, rule, args.seed)
        summaries.append(save_outputs(model, name, args.seed, args.animate))
    if args.demo:
        destination = ROOT / "outputs" / "comparison.json"
        destination.write_text(json.dumps(summaries, indent=2), encoding="utf-8")
    print("Saved results in:", ROOT / "outputs")
    print("Open plot.png to view paths and final cells.")
    if args.animate:
        print("Open animation.gif to watch the saved movement.")


if __name__ == "__main__":
    main()
```